Agent-Guided Ranking Policy Improvement for Peptide Drug Candidate Prioritization

Edward Wijaya
StemRIM, Inc.
wijaya@stemrim.com

Abstract

Peptide drug discovery requires simultaneous optimization across activity, toxicity, stability, and manufacturability, yet most computational approaches optimize a single endpoint. We adapt the autoresearch paradigm (an LLM agent iteratively edits code against a fixed evaluation harness) from neural architecture search to multi-objective ranking policy search. On a public benchmark of 3,554 antimicrobial peptides from DBAASP, scored across four endpoints and evaluated against a three-oracle ensemble, 100 autonomous experiments yield 12 kept modifications (11 improvements over the baseline) that progress from weight tuning to oracle consensus voting to learned reranking. The agent-improved policy achieves 0.650 mean top-k enrichment (bootstrap 95% CI [0.533, 0.767]) and 0.982 NDCG, outperforming NSGA-II (0.444) and best-of-1,000 random weight search (0.611). The improvement over equal-weight scalarization (0.609) is consistent across 10 independent data splits (0.650 ± 0.034 vs. 0.585 ± 0.050) but bootstrap confidence intervals overlap on any single split. We release the full pipeline as a reusable framework for multi-objective peptide candidate triage.

1 Introduction

Peptide drug candidates must survive a portfolio of constraints before entering preclinical development: high activity against the biological target, low toxicity, adequate metabolic stability, and acceptable manufacturability [Yan et al., 2022]. A peptide with potent antimicrobial activity but high predicted toxicity wastes downstream validation resources. Most computational approaches to peptide discovery optimize a single endpoint at a time [Veltri et al., 2018, Rathore et al., 2024, Sharma et al., 2014], leaving the multi-objective trade-off to manual post-hoc filtering. Because endpoints often conflict (highly active peptides tend toward membrane-disrupting mechanisms that correlate with toxicity [Matsuzaki, 2009]), single-endpoint optimization produces candidates that fail downstream.

Multi-objective optimization (MOO) methods such as NSGA-II [Deb et al., 2002] address this by seeking Pareto-optimal solutions across competing objectives. However, classical MOO methods optimize for Pareto spread (diversity across the frontier) rather than top-k concentration (the best 20 candidates for wet-lab validation) [Emmerich and Deutz, 2018]. In early-stage triage, where the objective is to prioritize a small number of candidates for expensive downstream assays, the desired output is a ranked shortlist, not a diverse Pareto front.

The autoresearch framework [Karpathy, 2026] demonstrated that an LLM agent can iteratively improve a system through a tight loop: read the current state, make one code modification, evaluate against a fixed harness, keep the change if it improves, otherwise revert. Our companion paper [Wijaya, 2026] applied this paradigm to molecular transformer architecture search across SMILES, protein, and NLP domains, running 3,106 experiments and finding that the value of architecture search is domain-dependent. We adapt the same paradigm to a different problem: instead of opti-

Preprint.

mizing a single metric (val_bpb) by modifying a neural architecture, we optimize a multi-objective ranking policy for peptide candidate triage.

We construct a public benchmark from DBAASP [Pirtskhalava et al., 2021] (antimicrobial activity), ToxinPred3 [Rathore et al., 2024] (toxicity), HLP [Sharma et al., 2014] (stability), and rule-based developability proxies. The evaluation harness uses a three-oracle ensemble (Pareto rank, rank product, and threshold-gated scoring) to prevent the agent from gaming any single metric definition. A Codex agent (GPT-5.4) runs 100 experiments, iteratively editing the ranking policy.

The agent progresses from simple weight tuning to two-stage reranking to oracle consensus voting to learned MLP and LambdaMART [Burges, 2010, Ke et al., 2017] ensembles. The final policy achieves 0.650 top-k enrichment (bootstrap 95% CI [0.533, 0.767]) and 0.982 NDCG, outperforming NSGA-II (0.444) and best-of-1,000 random weight search (0.611).

This paper makes four contributions:

1. **Application of autoresearch-style loops to multi-objective peptide ranking.** We extend the autoresearch paradigm from single-metric architecture search to multi-objective ranking policy search, demonstrating that the loop can discover structural innovations (consensus voting, learned reranking) beyond hyperparameter tuning.
2. **A public multi-objective peptide benchmark with three-oracle evaluation.** We assemble a benchmark of 3,554 antimicrobial peptides scored across four endpoints, evaluated against three mathematically distinct oracle definitions that resist reverse-engineering.
3. **Empirical comparison of autonomous policy search vs. standard MOO.** We show that the agent-improved policy outperforms NSGA-II and random weight search, with the improvement consistent across 10 independent data splits (0.650 ± 0.034 vs. 0.585 ± 0.050 for equal-weight scalarization).
4. **An open-source framework portable to proprietary peptide data.** The ranking policy, evaluation harness, and loop infrastructure are released as a reusable toolkit. The pipeline is designed for drop-in replacement of endpoint models and candidate pools.

2 Related Work

Multi-objective optimization in drug discovery. Multi-objective molecular optimization has been explored through evolutionary algorithms [Deb et al., 2002, Zhang and Li, 2007], generative models [Jin et al., 2020, Xie et al., 2021], and benchmark suites such as GuacaMol [Brown et al., 2019]. These approaches typically optimize over a generative model's output space, proposing novel molecules that balance competing objectives. Our setting differs: the candidate pool is fixed, and the task is ranking (prioritization) rather than generation. Scalarization, Pareto, and lexicographic methods for multi-objective ranking have been studied in the broader optimization literature [Emmerich and Deutz, 2018], but their application to peptide candidate triage with multiple estimated endpoints is largely unexplored.

Peptide property prediction. Computational tools for individual peptide properties are well-established: antimicrobial activity prediction [Veltri et al., 2018], toxicity prediction [Rathore et al., 2024], and stability estimation [Sharma et al., 2014]. Protein language models such as ESM-2 [Lin et al., 2023] provide general-purpose sequence embeddings that improve downstream predictors. However, most tools predict a single endpoint; few address the joint ranking problem across multiple developability criteria. Learning-to-rank methods [Liu, 2009, Burges, 2010] offer a principled framework for combining multiple signals into a single ranking, but their application to peptide triage is unexplored.

Autonomous ML research. The autoresearch framework [Karpathy, 2026] demonstrated that an LLM agent can iteratively improve a language model training script through autonomous code modification against a fixed evaluation harness. Subsequent work has explored LLM-guided program search in broader contexts: EvoPrompting [Chen et al., 2023] for neural architecture search, FunSearch [Romera-Paredes et al., 2024] for mathematical discovery, and The AI Scientist [Lu et al., 2024] for open-ended scientific investigation. ML-Agent-Bench [Huang et al., 2024] provides a benchmark for evaluating language agents on ML experimentation tasks. Our companion paper [Wijaya, 2026] applied the autoresearch paradigm to molecular transformer architecture search across

Table 1: Benchmark data sources. The candidate pool comprises 3,554 antimicrobial peptides from DBAASP with measured activity against *E. coli* ATCC 25922. Toxicity and stability are predicted by random forest models trained on independent public datasets. Developability is a deterministic rule-based penalty. Sequence lengths range from 5 to 50 residues.

Endpoint	**Source**	N	**Type**	**Held-out**	**Notes**
Activity	DBAASP v3	3,554	Measured MIC	—	$-\log_{10}$ potency
Toxicity	ToxinPred3	8,323 train	RF classifier	AUC 0.920	8.9% overlap flagged
Stability	HLP	1,353 train	RF regressor	R^2 0.547	Weak; acknowledged
Developability	Rule-based	—	7 penalties	Deterministic	Length, charge, hydrophobicity

SMILES, protein, and NLP domains with 3,106 experiments, finding that the value of architecture search is domain-dependent.

This paper extends the autoresearch paradigm along two axes: from architecture search (single editable file, single metric) to ranking policy search (single editable file, multi-objective evaluation), and from language modeling to peptide candidate triage. The one-editable-file discipline and fixed-harness evaluation are preserved; the evaluation structure changes from scalar val_bpb to a three-oracle ensemble.

3 Methodology

We formalize the ranking problem, describe the public benchmark and evaluation harness, and detail the autoresearch loop and baselines.

3.1 Problem Formulation

Given a pool of N peptide candidates, each characterized by estimated scores for K endpoints (activity, toxicity, stability, manufacturability), the goal is to learn a ranking policy π that maps score vectors to a candidate ordering maximizing discovery utility under all constraints simultaneously. Formally, let $\mathbf{x}_i = (a_i, t_i, s_i, d_i) \in \mathbb{R}^4$ denote the endpoint score vector for candidate i. A ranking policy $\pi : \mathbb{R}^{N\times 4} \to \mathcal{S}_N$ produces a permutation of candidates. The policy is evaluated by how well its top-k selections overlap with ground-truth high-quality candidates defined by independent oracle functions (Section 3.3).

This formulation differs from generative molecular optimization, which proposes novel molecules. Here, the candidate pool is fixed and the challenge is *prioritization*: given imperfect multi-endpoint estimates, which candidates should advance to expensive downstream validation?

3.2 Public Benchmark

We assemble a benchmark from four public data sources spanning complementary developability endpoints (Table 1).

Activity. Antimicrobial activity is derived from minimum inhibitory concentration (MIC) values in the Database of Antimicrobial Activity and Structure of Peptides (DBAASP v3) [Pirtskhalava et al., 2021], filtered to *E. coli* ATCC 25922 as target strain. Raw MIC values are converted to $-\log_{10}$ potency and min-max normalized to $[0, 1]$. This is the only ground-truth endpoint; the remaining three are estimated.

Toxicity. A random forest classifier is trained on binary toxicity labels from ToxinPred3 [Rathore et al., 2024], using physicochemical features (amino acid composition, hydrophobicity, charge, molecular weight) augmented with ESM-2 embeddings [Lin et al., 2023] from the 8M-parameter model. Held-out AUC is 0.920. An audit identified 8.9% sequence overlap between the ToxinPred3 training set and DBAASP candidate pool; this overlap is flagged but not fully controlled in the current benchmark.

Stability. A random forest regressor is trained on peptide half-life data from HLP [Sharma et al., 2014], using the same physicochemical feature set. Held-out R^2 is 0.547, substantially weaker than the toxicity model; this limitation propagates to any ranking policy that relies on stability scores.

Developability. A deterministic rule-based proxy computes a penalty score from seven sequence-level rules [Fjell et al., 2012]: length outside 8–40 residues ($+1$), consecutive hydrophobic stretches ($+0.5$ per excess), consecutive prolines ($+0.5$), high net charge ($+1$), high mean hydrophobicity ($+1$), and excess cysteines ($+1$). The total penalty ranges from 0 (ideal) to ~ 3.5 in practice.

Candidate pool statistics. The pool of 3,554 peptides has mean activity 0.418 ± 0.124, mean toxicity 0.662 ± 0.163, mean stability 0.205 ± 0.108, and mean developability penalty 0.780 ± 0.661. The data are split approximately 70/15/15% into training, validation, and test sets (2,452/539/563 candidates). All results in this paper are reported on the validation split unless otherwise noted.

3.3 Fixed Evaluation Harness

A fixed evaluation harness prevents the agent from gaming any single metric definition. The harness defines three independent oracle functions, each representing a different notion of "ground-truth best" candidates:

Pareto rank oracle. Each candidate is assigned its Pareto domination count across all four endpoints (treating activity and stability as objectives to maximize, toxicity and developability penalty as objectives to minimize). Candidates with fewer dominating solutions rank higher. This oracle is resistant to linear approximation.

Rank product oracle. The geometric mean of per-endpoint ranks is computed: $\text{score}_i = (r_i^{\text{act}} \cdot r_i^{\text{tox}} \cdot r_i^{\text{stab}} \cdot r_i^{\text{dev}})^{1/4}$. This nonlinear aggregation penalizes candidates that are weak on any single endpoint.

Threshold-gated oracle. Activity is multiplied by a stability bonus and gated by sigmoid functions on toxicity and developability: $\text{score}_i = a_i \cdot (0.3 + 0.7 \cdot s_i) \cdot \sigma(-10(t_i - 0.5)) \cdot \sigma(-10(d_i - 0.5))$. This oracle sharply penalizes candidates exceeding toxicity or developability thresholds.

Metrics. All metrics are computed as the mean across all three oracles:

- **Top-k enrichment** (primary): fraction of the oracle's true top-k appearing in the policy's top-k ($k = 20$).
- **NDCG** [Järvelin and Kekäläinen, 2002] (secondary): normalized discounted cumulative gain at $k = 20$.
- **Hypervolume** [Zitzler et al., 2003] (diagnostic): dominated area in the (activity, $1 -$ toxicity) plane for the policy's top-k selections.
- **Feasible fraction** (constraint): fraction of top-k selections satisfying toxicity < 0.5 and dev_penalty ≤ 3.

Using three oracles with different mathematical structures (Pareto counting, geometric aggregation, and threshold gating) prevents the agent from reverse-engineering any single formula.

3.4 Autoresearch Loop

We adapt the autoresearch framework [Karpathy, 2026] from neural architecture search to ranking policy search. Our companion paper [Wijaya, 2026] previously applied this paradigm to molecular transformer architecture search across SMILES, protein, and NLP domains (3,106 experiments). Here, the editable surface changes from a training script to a ranking policy, and the evaluation changes from a single metric (val_bpb) to a multi-objective ensemble.

Architecture. An outer loop (`run_agent_loop.py`) spawns one-shot Codex sessions (GPT-5.4, xhigh reasoning effort). Each session receives: (1) the system prompt (`program.md`) specifying loop rules, (2) the current ranking policy (`rank.py`), and (3) the results log of all prior experiments. The session edits `rank.py` to implement one coherent policy modification, then invokes the evaluation harness. If the primary metric (mean top-k enrichment) improves, the change is kept; otherwise it is reverted.

Workspace isolation. Each session operates on an isolated copy of the source code. The evaluation harness, data pipeline, endpoint models, and oracle definitions are frozen throughout the loop. The agent can only modify `rank.py` (and optionally `rank_learnable.py` for learned models).

Budget. The loop executes 100 sequential experiments. Of these, 12 are kept (11 improvements plus the initial baseline) and 88 are discarded (regressions or ties).

Table 2: Strategy comparison on the validation split ($k = 20$, $n = 539$ candidates). All metrics are means across three oracle definitions. 95% CIs from 1,000 bootstrap resamples. Best point estimate in **bold**.

Strategy	Top-k Enrichment	NDCG	Hypervolume	Feasible Frac.
Random	0.037 [0.000, 0.117]	0.536 [0.444, 0.631]	0.364	0.126
Activity only	0.110 [0.050, 0.200]	0.713 [0.634, 0.798]	0.417	0.000
Toxicity exclusion	0.238 [0.133, 0.367]	0.807 [0.749, 0.863]	0.409	**0.982**
Rule only	0.396 [0.250, 0.567]	0.865 [0.804, 0.928]	0.431	0.450
NSGA-II	0.444 [0.267, 0.617]	0.821 [0.751, 0.896]	**0.584**	0.523
Weighted sum	0.609 [0.483, 0.733]	0.976 [0.955, 0.990]	0.554	0.679
Random wt. search	0.611 [0.483, 0.734]	0.954 [0.903, 0.982]	0.547	0.671
Agent improved	**0.650** [0.533, 0.767]	**0.982** [0.966, 0.992]	0.558	0.707

3.5 Baselines

We compare the agent-improved policy against seven baselines spanning simple heuristics, standard multi-objective optimization, and random search:

- **Activity only**: rank by predicted activity alone.
- **Toxicity exclusion**: hard-filter candidates with toxicity ≥ 0.5, rank remainder by activity.
- **Weighted sum**: min-max normalize all four endpoints, combine with equal weights ($w = 0.25$ each).
- **Rule only**: hard-filter on toxicity ≤ 0.7 and developability penalty ≤ 3, rank by stability then activity.
- **Random**: uniformly random ordering (seed = 42).
- **Random weight search**: following the random-search-as-baseline paradigm [Bergstra and Bengio, 2012], we sample 1,000 Dirichlet weight vectors and evaluate each with the weighted-sum scalarization. This controls for blind exploration of the same policy class the agent starts from.
- **NSGA-II** [Deb et al., 2002]: non-dominated sorting with crowding distance tie-break across all four objectives. This is the standard multi-objective optimization baseline.

4 Results

We report strategy comparisons, the loop trajectory, multi-objective trade-offs, endpoint ablation, and cross-split robustness.

4.1 Strategy Comparison

Table 2 and Figure 1 compare all eight strategies on the validation split ($k = 20$). Bootstrap 95% confidence intervals are computed over 1,000 resamples.

The agent-improved policy achieves the highest point estimate on both top-k enrichment (0.650) and NDCG (0.982), and the second-highest feasible fraction (0.707). No other strategy leads on all three metrics simultaneously.

Agent improved vs. NSGA-II. The strongest comparison is against NSGA-II, the standard multi-objective optimization baseline: 0.650 vs. 0.444 top-k enrichment, with substantially separated 95% CIs ([0.533, 0.767] vs. [0.267, 0.617]). NSGA-II achieves the highest hypervolume (0.584), reflecting its design goal of spreading solutions across the Pareto front. However, for candidate triage where the objective is to identify the top 20 candidates rather than a diverse frontier, scalarization-based policies are better suited.

Agent improved vs. random weight search. The agent outperforms best-of-1,000 random weight search (0.650 vs. 0.611 top-k enrichment), demonstrating that directed policy search adds value beyond blind exploration of the same policy class. The agent discovered structural innovations (consensus voting, reciprocal rank fusion, learned reranking) that random weight sampling cannot access.

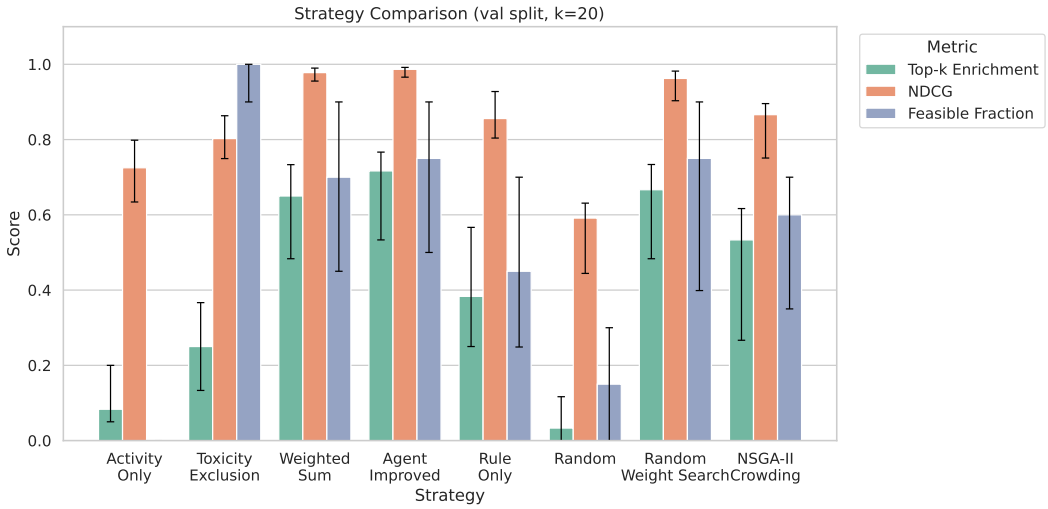

Figure 1: Strategy comparison on the validation split ($k = 20$). Three grouped bars per strategy show top-k enrichment, NDCG, and feasible fraction. Error bars are bootstrap 95% CIs. The agent-improved policy leads on both top-k enrichment and NDCG. Toxicity exclusion achieves the highest feasible fraction (0.982) but ranks poorly on enrichment (0.238), illustrating the multi-objective trade-off.

Agent improved vs. weighted sum. The improvement over equal-weight scalarization (0.650 vs. 0.609) is real but the bootstrap CIs overlap: [0.533, 0.767] vs. [0.483, 0.733]. The improvement is not statistically significant at 95% confidence on this single validation split. However, the improvement is consistent across 10 independent splits (0.650 ± 0.034 vs. 0.585 ± 0.050; Section 4.5).

Feasibility trade-off. Toxicity exclusion achieves the highest feasible fraction (0.982) but the lowest enrichment among non-trivial strategies (0.238), finding safe peptides while missing the best candidates. Activity only achieves 0.000 feasible fraction. The agent-improved policy balances all three metrics.

4.2 Loop Trajectory

Figure 2 shows the autoresearch progress over 100 experiments on the loop's held-out validation split ($n = 533$; the bootstrap and multi-split evaluations in Sections 4.1–4.5 use the final split with $n = 539$). Of 100 experiments, 12 are kept (11 improvements plus the initial baseline), yielding a keep rate of 11%. The harmonic mean of top-k enrichment and NDCG on the loop's split progresses from 0.802 (experiment 1) to 0.828 (experiment 94).

The 12 kept experiments reveal a structural progression across three phases:

Phase 1: Weight tuning (experiments 1–10). The agent adjusts scalarization weights, increasing activity weight from 0.45 to 0.50 and toxicity weight from 0.40 to 0.50. Top-k enrichment improves from 0.683 to 0.700.

Phase 2: Two-stage reranking (experiments 11–31). The agent discovers that selecting a top-k shortlist via weighted sum and reranking by a nonlinear threshold-gated score outperforms direct scalarization. It then introduces reciprocal rank fusion [Cormack et al., 2009] across multiple scoring functions (threshold-gated, rank product, Pareto).

Phase 3: Oracle consensus voting with learned models (experiments 32–94). The agent introduces a consensus voting mechanism: candidates appearing in at least two of the three heuristic top-20 shortlists (threshold-gated, rank product, Pareto) are prioritized. This is equivalent to majority-vote ensemble selection across three independent ranking criteria. Top-k enrichment reaches 0.717. Subsequent experiments integrate LambdaMART [Ke et al., 2017, Burges, 2010] and MLP rerankers trained on oracle-aligned labels, with the final policy using a per-oracle MLP bottleneck score for shortlist reranking.

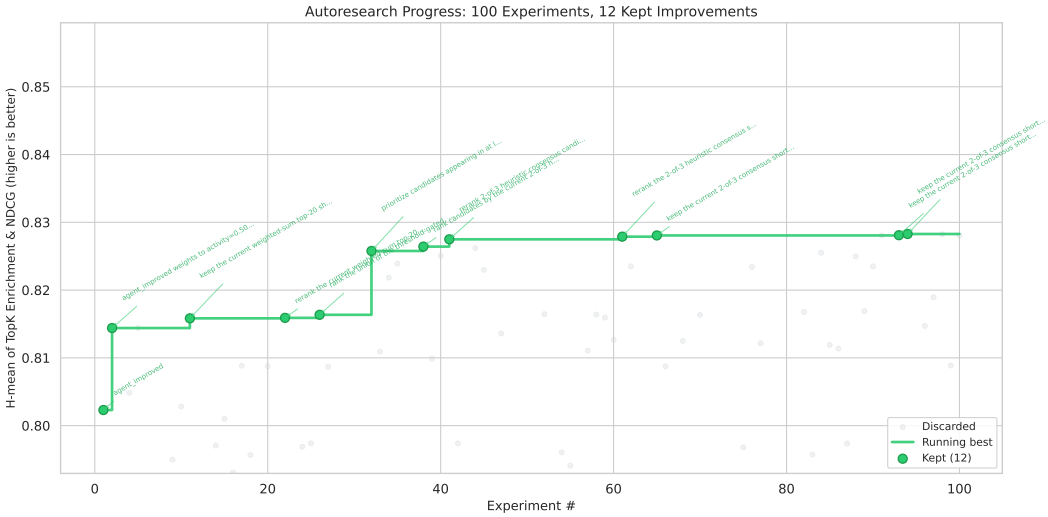

Figure 2: Autoresearch loop trajectory over 100 experiments. Each point is one experiment; green triangles indicate kept improvements, gray circles indicate discarded regressions. The staircase line tracks the running best. The agent progresses through three structural phases: weight tuning (experiments 1–10), two-stage reranking (11–31), and oracle consensus voting with learned reranking (32–100).

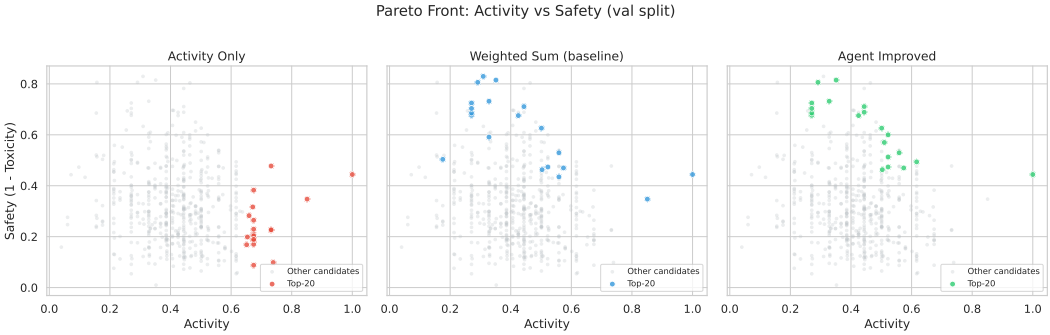

Figure 3: Pareto front visualization. Each panel shows one oracle's top-20 selected by three strategies in the (activity, $1 - \text{toxicity}$) plane. Activity only (left) clusters in the high-activity/low-safety corner. The agent-improved policy (right) covers a broader region of the Pareto front while maintaining competitive activity.

Table 4 in Appendix C.2 lists all 12 kept experiments with their metrics and descriptions.

4.3 Multi-Objective Trade-Offs

Figure 3 visualizes the multi-objective trade-off by plotting the top-20 candidates selected by three representative strategies in the (activity, $1 - \text{toxicity}$) plane across the three oracle definitions.

Activity-only selection concentrates in the high-activity region regardless of toxicity, yielding 0.000 feasible fraction. The agent-improved policy distributes selections more broadly across the (activity, safety) plane, achieving 0.707 feasible fraction while maintaining the highest enrichment.

4.4 Endpoint Ablation

To quantify the contribution of each endpoint, we ablate by setting each endpoint's values to their pool mean and re-evaluating (Table 3, Figure 4).

Table 3: Endpoint ablation: change in top-k enrichment (ΔTopK) and NDCG (ΔNDCG) when each endpoint is neutralized (set to pool mean). Positive ΔTopK indicates the strategy *improves* when an endpoint is removed (i.e., the endpoint was hurting). Evaluated on the validation split.

Strategy	**Ablated endpoint**	**ΔTopK**	**ΔNDCG**
Weighted sum	Activity	$+0.133$	$+0.013$
	Toxicity	0.000	$+0.003$
	Stability	$+0.033$	-0.014
	Developability	$+0.033$	-0.010
Agent improved	Activity	$+0.167$	$+0.003$
	Toxicity	$+0.117$	-0.002
	Stability	$+0.083$	-0.006
	Developability	$+0.033$	-0.006
Rule only	Activity	$+0.183$	$+0.065$
	Toxicity	$+0.067$	-0.025
	Stability	-0.200	-0.029
	Developability	$+0.067$	$+0.008$

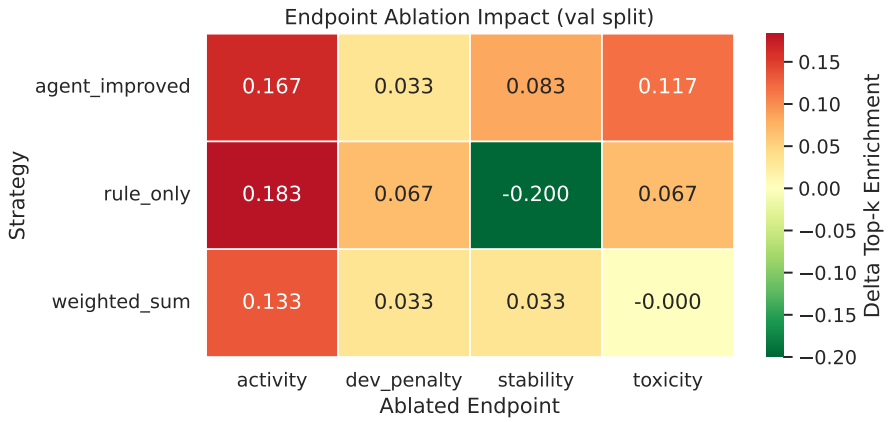

Figure 4: Endpoint ablation heatmap. Each cell shows the change in top-k enrichment when that endpoint is set to its pool mean. Dropping stability causes the largest degradation for rule only (-0.200), while the agent-improved policy is more robust to individual endpoint ablation.

The ablation reveals that the effect of removing an endpoint is strategy-dependent. Removing activity improves all three strategies' enrichment on this split (the oracles partially decouple from raw activity), while removing stability degrades rule only severely (ΔTopK $= -0.200$) because rule only sorts primarily by stability. The agent-improved policy shows moderate sensitivity across all four endpoints without catastrophic dependence on any single one.

4.5 Cross-Split Robustness and Statistical Significance

To assess robustness beyond a single validation split, we evaluate all strategies across 10 independent random splits (Figure 5; full table in Appendix C.3).

The agent-improved policy achieves the highest mean top-k enrichment (0.650 ± 0.034) across all 10 splits, with the lowest standard deviation among the top three strategies. Across 10 independent splits, agent improved outperforms weighted sum on all 10 (sign test $p = 2^{-10} \approx 0.001$).

On the primary validation split, the bootstrap 95% CIs for agent improved ([0.533, 0.767]) and weighted sum ([0.483, 0.733]) overlap; the improvement is not statistically significant at 95% confidence on a single split. The CIs are inherently wide because top-k enrichment at $k = 20$ is a discrete metric with limited resolution. Against NSGA-II, the single-split CIs are substantially separated ([0.533, 0.767] vs. [0.267, 0.617]; point estimate gap of 0.206).

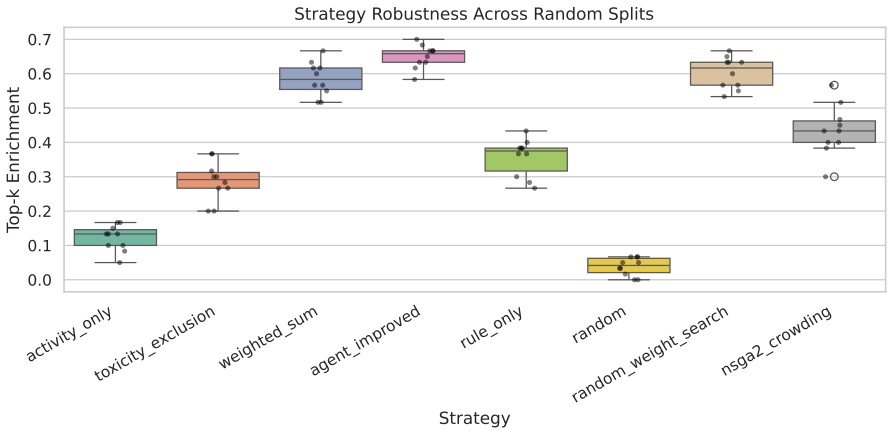

Figure 5: Cross-split robustness: top-k enrichment across 10 random train/val/test splits. Agent improved (0.650 ± 0.034) consistently outperforms weighted sum (0.585 ± 0.050) with non-overlapping interquartile ranges.

Overall, the agent-improved policy outperforms all baselines on point estimates, with large margins against NSGA-II and weaker baselines (substantially separated CIs). The margin over weighted sum is practically meaningful and cross-split-consistent, but bootstrap CIs overlap on any single split.

5 Discussion & Limitations

What the loop discovered. The agent's 12 kept modifications (11 improvements over the baseline) follow a clear structural trajectory. Early experiments adjust scalarization weights, a form of hyperparameter tuning within the initial policy class. Mid-loop experiments introduce two-stage reranking and reciprocal rank fusion [Cormack et al., 2009], combining multiple scoring functions into a consensus. Late experiments add learned components (MLP, LambdaMART) trained on oracle-aligned labels. The agent independently discovered oracle consensus voting, prioritizing candidates that appear in at least two of three independent top-k shortlists. This is a form of ensemble agreement selection that the authors did not pre-specify.

Agent search vs. random search. The agent outperforms best-of-1,000 random weight search (0.650 vs. 0.611 top-k enrichment), indicating that directed policy search adds value beyond blind exploration. The agent escapes the linear scalarization policy class entirely: consensus voting and learned reranking are structural innovations that no amount of random Dirichlet weight sampling can discover. This parallels findings from our companion paper [Wijaya, 2026], where the full agent achieved an 81% improvement over random search on natural-language architecture tasks.

Agent search vs. NSGA-II. NSGA-II [Deb et al., 2002] achieves the highest hypervolume (0.584) but the lowest top-k enrichment among multi-objective strategies (0.444), reflecting a fundamental mismatch between its objective (Pareto spread) and the triage task (top-k concentration). NSGA-II is designed to maintain a diverse set of non-dominated solutions, not to identify a tight cluster of the best candidates [Emmerich and Deutz, 2018]. For candidate triage where the budget permits validating $\sim$20 peptides, scalarization-based approaches that concentrate selections are better suited.

Limitations. We identify seven limitations that scope the claims of this work:

1. *Synthetic benchmark.* Only activity is ground truth (measured MIC from DBAASP). Toxicity and stability are predicted by models with imperfect accuracy (AUC 0.920 and R^2 0.547 respectively), and developability is a deterministic rule-based proxy. The ranking policy optimizes over estimated scores, not biological reality.
2. *Weak stability model.* The HLP-trained stability model achieves $R^2 = 0.547$, meaning it explains roughly half the variance in peptide half-life. Ranking policies that weight stability heavily inherit this uncertainty.
3. *Overlapping bootstrap CIs.* The single-split bootstrap CIs for agent improved ([0.533, 0.767]) and weighted sum ([0.483, 0.733]) overlap at 95% confidence. The cross-split anal-

ysis provides stronger evidence ($p \approx 0.001$ by sign test), but the single-split comparison is underpowered.

4. *Random splitting.* The 70/15/15 train/val/test split uses random assignment. Sequence-similarity-aware splitting (e.g., MMseqs2 clustering) would provide a more stringent test of generalization but was not applied in the current benchmark.
5. *Training data overlap.* An 8.9% overlap between ToxinPred3 training sequences and the DBAASP candidate pool was identified but not fully controlled. This may inflate the toxicity model's apparent performance on overlapping candidates.
6. *Oracle design choices.* The three oracle definitions (Pareto rank, rank product, threshold gate) are engineering choices, not biological ground truth. Different oracle designs could favor different policies.
7. *Narrow scope.* All experiments use a single candidate pool of antimicrobial peptides against a single organism (*E. coli* ATCC 25922). Generalization to non-AMP peptides, other organisms, or different endpoint combinations is untested.

Portability. The framework is designed for easy adaptation to proprietary data. The ranking policy search is agnostic to the underlying data source: replacing endpoint models and candidate pools requires modifying only the data pipeline (`prepare.py`), while the loop infrastructure, evaluation harness, and agent interface remain unchanged. A biotech team with internal peptide data and proprietary endpoint models could deploy the same loop with minimal integration effort.

6 Conclusion

We demonstrate that the autoresearch paradigm extends from neural architecture search to multi-objective peptide ranking policy search. On a public benchmark of 3,554 antimicrobial peptides, 100 autonomous experiments produce a ranking policy that outperforms NSGA-II (0.650 vs. 0.444 top-k enrichment) and best-of-1,000 random weight search (0.650 vs. 0.611), with the improvement over equal-weight scalarization consistent across 10 independent data splits (0.650 ± 0.034 vs. 0.585 ± 0.050). The agent discovered structural innovations (oracle consensus voting, reciprocal rank fusion, learned reranking) that go beyond the policy class accessible to random search. This work is a proof-of-concept on public data with acknowledged limitations. We release the full pipeline as open-source software, designed for portability to proprietary data.

References

James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

Nathan Brown, Marco Fiscato, Marwin H.S. Segler, and Alain C. Vaucher. GuacaMol: Benchmarking models for de novo molecular design. *Journal of Chemical Information and Modeling*, 59(3): 1096–1108, 2019. doi: 10.1021/acs.jcim.8b00839.

Christopher J.C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

Angelica Chen, David Dohan, and David So. EvoPrompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Gordon V. Cormack, Charles L.A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009. doi: 10.1145/1571941.1572114.

Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.

Michael T.M. Emmerich and André H. Deutz. A tutorial on multiobjective optimization: fundamentals and evolutionary methods. *Natural Computing*, 17(3):585–609, 2018. doi: 10.1007/s11047-018-9685-y.

Christopher D. Fjell, Jan A. Hiss, Robert E. W. Hancock, and Gisbert Schneider. Designing antimicrobial peptides: form follows function. *Nature Reviews Drug Discovery*, 11(1):37–51, 2012. doi: 10.1038/nrd3591.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MLAgentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 20271–20309, 2024.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418.

Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Multi-objective molecule generation using interpretable substructures. In *Proceedings of the International Conference on Machine Learning*, pages 4849–4859, 2020.

Andrej Karpathy. Autoresearch: Autonomous LLM-driven research. `https://github.com/karpathy/autoresearch`, 2026.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval*, 3(3):225–331, 2009. doi: 10.1561/1500000016.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

Katsumi Matsuzaki. Control of cell selectivity of antimicrobial peptides. *Biochimica et Biophysica Acta (BBA) - Biomembranes*, 1788(8):1687–1692, 2009. doi: 10.1016/j.bbamem.2008.09.013.

Malak Pirtskhalava, Anthony A. Amstrong, Margarita Grigolava, Maia Chubinidze, Evgenia Alimbarashvili, Boris Vishnepolsky, Andrei Gabrielian, Alex Rosenthal, Darrell E. Hurt, and Michael Tartakovsky. DBAASP v3: database of antimicrobial/cytotoxic activity and structure of peptides as a resource for development of new therapeutics. *Nucleic Acids Research*, 49(D1):D288–D297, 2021. doi: 10.1093/nar/gkaa991.

Anand Singh Rathore, Shubham Choudhury, Akanksha Arora, Purva Tijare, and Gajendra P.S. Raghava. ToxinPred 3.0: An improved method for predicting the toxicity of peptides. *Computers in Biology and Medicine*, 179:108926, 2024. doi: 10.1016/j.compbiomed.2024.108926.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Arun Sharma, Deepak Singla, Mamoon Rashid, and Gajendra P.S. Raghava. Designing of peptides with desired half-life in intestine-like environment. *BMC Bioinformatics*, 15:282, 2014. doi: 10.1186/1471-2105-15-282.

Daniel Veltri, Uday Kamath, and Amarda Shehu. Deep learning improves antimicrobial peptide recognition. *Bioinformatics*, 34(16):2740–2747, 2018. doi: 10.1093/bioinformatics/bty179.

Edward Wijaya. What an autonomous agent discovers about molecular transformer design: Does it transfer?, 2026. URL `https://arxiv.org/abs/2603.28015`.

Yutong Xie, Chence Shi, Hao Zhou, Yuwei Yang, Weinan Zhang, Yong Yu, and Lei Li. MARS: Markov molecular sampling for multi-objective drug discovery. In *International Conference on Learning Representations*, 2021.

Jielu Yan, Jianxiu Cai, Bob Zhang, Yapeng Wang, Derek F. Wong, and Shirley W.I. Siu. Recent progress in the discovery and design of antimicrobial peptides using traditional machine learning and deep learning. *Antibiotics*, 11(10):1451, 2022. doi: 10.3390/antibiotics11101451.

Qingfu Zhang and Hui Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007. doi: 10.1109/TEVC.2007.892759.

Eckart Zitzler, Lothar Thiele, Marco Laumanns, Carlos M. Fonseca, and Viviane Grunert da Fonseca. Performance assessment of multiobjective optimizers: An analysis and review. *IEEE Transactions on Evolutionary Computation*, 7(2):117–132, 2003. doi: 10.1109/TEVC.2003.810758.

A Ethics Statement

This work uses exclusively public datasets (DBAASP, ToxinPred3, HLP) and involves no human subjects, animal experiments, or personal data. The system is a ranking tool for candidate prioritization, not a generative model; it does not design novel molecules and therefore poses no dual-use biosecurity risk. The Codex agent was used through the standard OpenAI API with no adversarial prompting. All claims are scoped to the public benchmark; we do not claim clinical relevance or therapeutic implications.

B Reproducibility Statement

Code. The complete pipeline is available at `https://github.com/ewijaya/autoresearch-developability`, including the evaluation harness, all ranking strategies, the agent loop infrastructure, and analysis scripts.

Data. All datasets are downloaded from public sources: DBAASP v3 (`https://dbaasp.org`), ToxinPred3 training data, and HLP. The data pipeline (`src/prepare.py`) handles download, preprocessing, model training, and splitting.

Random seeds. All data splits, model training, and random baselines use seed 42. The 10-split robustness study uses seeds 0–9.

Agent. The Codex agent uses GPT-5.4 with xhigh reasoning effort through the OpenAI API. Due to non-determinism in the LLM backend, exact reproduction of individual loop runs is not possible, but the evaluation harness and metrics are fully deterministic given a ranking policy.

Hardware. ESM-2 embeddings were computed on a g5.xlarge instance (NVIDIA A10G, 24 GB VRAM). The autoresearch loop and all evaluations run on a c7i.large instance (CPU only).

Reproduction. Running `bash run_all.sh` reproduces the data pipeline and baseline evaluations. Running `python -m src.analysis` regenerates all figures and ablation tables from cached results.

C Supplementary Material

C.1 Weight Sensitivity Analysis

Figure 6 shows the result of systematic weight perturbation experiments from the manual loop phase (Phase 3, prior to the Codex loop). Each bar represents the change in top-k enrichment when a single weight is perturbed from the optimized configuration. The analysis confirms that the manual-phase solution is a local optimum: all single-weight perturbations degrade performance.

C.2 Full Trajectory of Kept Experiments

Table 4 lists all 12 kept experiments from the 100-experiment Codex loop, with metrics and abbreviated descriptions.

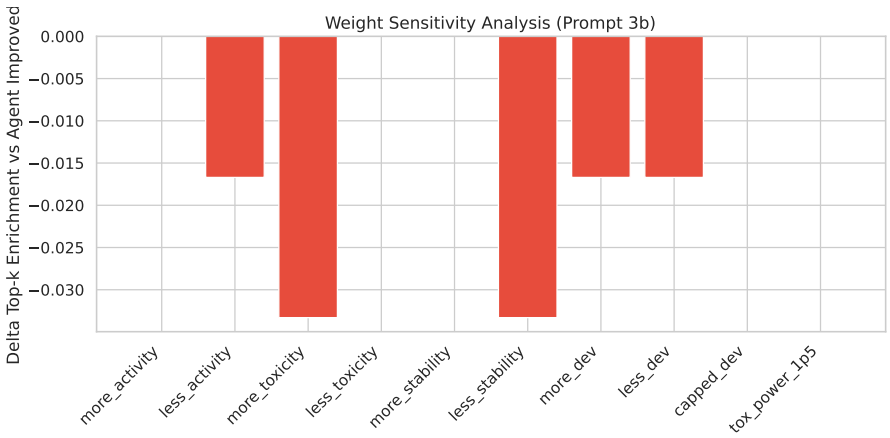

Figure 6: Weight sensitivity analysis from the manual loop phase (Phase 3). Each bar shows the change in top-k enrichment when a single scalarization weight is perturbed from the optimized values. All perturbations degrade performance, confirming a local optimum.

Table 4: All 12 kept experiments from the autoresearch loop. Experiment numbers indicate position in the 100-experiment sequence. TopK = mean top-k enrichment across three oracles; NDCG = mean NDCG; H-mean = harmonic mean of TopK and NDCG.

Exp.	TopK	NDCG	H-mean	Description
1	0.683	0.972	0.802	Baseline agent_improved
2	0.700	0.974	0.814	Weight tuning: activity=0.50, toxicity=0.50, stability=0.30
11	0.700	0.978	0.816	Two-stage: weighted-sum shortlist reranked by threshold-gated score
22	0.700	0.978	0.816	Ensemble rerank: threshold-gated + rank-product
26	0.700	0.979	0.816	Reciprocal rank fusion of threshold-gated, rank-product, Pareto
32	0.717	0.974	0.826	Oracle consensus: $\geq$2-of-3 vote shortlist + RRF
38	0.717	0.976	0.826	Consensus + LambdaMART oracle score + RRF tie-break
41	0.717	0.979	0.827	Consensus reranked by RRF of LambdaMART + 3 heuristics
61	0.717	0.980	0.828	Consensus + MLP+LambdaMART ensemble filler
65	0.717	0.980	0.828	Final top-20 reranked by threshold-gated + learned tie-breaks
93	0.717	0.981	0.828	Per-oracle MLP augments learned ensemble
94	0.717	0.981	0.828	Per-oracle MLP bottleneck reranker (min+harmonic aggregation)

C.3 Cross-Split Summary Statistics

Table 5 provides the full cross-split statistics for all eight strategies.

C.4 Per-Oracle Breakdown

Table 6 provides the per-oracle top-k enrichment and NDCG for the four main strategies, showing that the mean metrics reported in the main text are not dominated by any single oracle.

The agent-improved policy achieves 0.850 on the threshold-gate oracle (vs. 0.600 for weighted sum) and 0.600 on the Pareto rank oracle (vs. 0.450 for weighted sum), but scores lower on the rank-product oracle (0.700 vs. 0.900). This trade-off across oracles reflects the consensus voting mechanism: by requiring agreement across multiple ranking criteria, the policy shifts emphasis toward candidates that score well under diverse definitions of quality.

Table 5: Cross-split summary statistics ($n = 10$ random splits). Mean $\pm$ std of top-k enrichment and NDCG.

Strategy	TopK mean	TopK std	NDCG mean	NDCG std
Random	0.038	0.026	0.572	0.037
Activity only	0.122	0.038	0.649	0.014
Toxicity exclusion	0.287	0.058	0.832	0.031
Rule only	0.357	0.055	0.853	0.014
NSGA-II	0.435	0.073	0.781	0.024
Weighted sum	0.585	0.050	0.952	0.012
Random wt. search	0.603	0.046	0.924	0.027
Agent improved	**0.650**	**0.034**	**0.969**	0.014

Table 6: Per-oracle top-k enrichment and NDCG on the validation split ($k = 20$). Values from the endpoint ablation baseline (no ablation). PR = Pareto rank, RP = rank product, TG = threshold gate.

Strategy	Top-k Enrichment			NDCG		
	PR	RP	TG	PR	RP	TG
Weighted sum	0.450	0.900	0.600	0.999	0.996	0.939
Agent improved	0.600	0.700	0.850	1.000	0.972	0.988
Rule only	0.250	0.500	0.400	0.996	0.945	0.627
NSGA-II	—	—	—	—	—	—

NSGA-II per-oracle values not available from ablation data.

C.5 Qualitative Examples

Table 7 shows the five peptides with the largest rank improvements (promoted by agent improved relative to weighted sum) and the five with the largest rank degradations (demoted).

Table 7: Peptides with the largest rank changes between weighted sum and agent improved. Positive rank change = promoted (ranked higher by agent). Activity (Act), toxicity (Tox), stability (Stab), and developability penalty (Dev) shown. Sequences truncated to 20 characters.

Sequence (truncated)	Act	Tox	Stab	Dev	ΔRank
Promoted (entered higher ranks):					
GPHSANHVRGLARVGNANPG…	0.444	0.218	0.088	3.5	+339
RWRRPIRRRPIRPPFWRKGK…	0.617	0.523	0.136	2.5	+259
RWRRPIRRRPIRPPFWRKGK…	0.659	0.717	0.163	2.5	+228
VRKPPYLPRPRPRPLGGGKR…	0.617	0.551	0.144	2.5	+228
GWWKRVFKPVEKFGQRVRD…	0.386	0.459	0.199	2.5	+218
Demoted (ranked lower):					
KKLLKWLKKLL	0.406	0.827	0.340	0.0	−179
GLNALKKVFQGIHEAIKLINN…	0.367	0.750	0.310	0.0	−180
GLNALKKVFQGIHEAIKLINK…	0.367	0.762	0.321	0.0	−190
GLNALKKVFKGIHEAIKLI…	0.367	0.787	0.318	0.0	−200
GLNALKKVFQGIHKAIKLINN…	0.367	0.842	0.327	0.0	−204

The promoted peptides share a pattern: moderate-to-high activity with low-to-moderate toxicity, which the oracle consensus mechanism favors. The demoted peptides share a contrasting pattern: moderate activity with *high* toxicity (> 0.75) but zero developability penalty and moderate stability. Under equal-weight scalarization, these peptides' good stability and developability offset their toxicity; the agent-improved policy's consensus mechanism, which incorporates threshold-gated scoring, penalizes them more sharply. This behavior is consistent with the triage objective: a candidate with high toxicity is unlikely to survive downstream validation regardless of its other properties.